

PrivHSD System Design Paper

Date: 2026-06-11

Audience: project group, ML/privacy reviewer, and future implementation audit.

Executive Summary

PrivHSD is a row-preserving text privatization pipeline for hate-speech detection datasets. The goal is not to build the hate-speech classifier. The goal is to transform text so author-identifying signals are reduced while the signals needed for hate-speech detection remain usable.

The current recommendation is to pause major new implementation work until the official data arrives. The system is already strong enough for a first leaderboard attempt and for a methodology discussion. More local experiments will mostly refine proxies rather than improve the unknown official metric.

The default submission path is:

```
CSV rows
-> deterministic privacy masking
-> optional style normalization
-> exact-format CSV validation
-> manifest with hashes, columns, metrics, and git commit
```

The stronger experimental path is:

```
CSV rows
-> generate row-local candidates
-> score privacy/HSD tradeoff
-> choose one candidate per row
-> exact-format validation when used for submission
```

Optional Presidio, DPMLM, and local LLM outputs are candidates or evaluators. They are not trusted as direct replacements.

Scope

In Scope

- Preserve row count, row order, IDs, labels, authors, and metadata.
- Mask direct identifiers and conservative quasi-identifiers.
- Preserve hate-speech target/action/negation/modality cues by default.
- Reduce author-style signals with optional deterministic normalization.
- Produce audit JSON and exact-format submission manifests.
- Run local utility/privacy proxy checks before official scoring.

Out Of Scope Until Official Data

- Training a new end-to-end anonymization model.
- Treating DPMLM as the default output generator.
- Optimizing against local proxy metrics after they stop changing decisions.

- Claiming formal anonymity from local metrics.

Expected Input And Output

The official dataset is expected to be similar to:

Column	Meaning	Pipeline treatment
`id`	Row identifier	Preserved exactly. Also scanned for leakage into text.
`author`	Author/user label, if present	Preserved exactly. Used only for optional author-risk evaluation.
`text`	Source text to privatize	Transformed into `privatized_text` or replaced in place for exact submissions.
`HS` or `label`	Hate-speech label	Preserved exactly. Used only for local utility evaluation.
Extra columns	Metadata	Preserved exactly unless explicitly declared text columns.

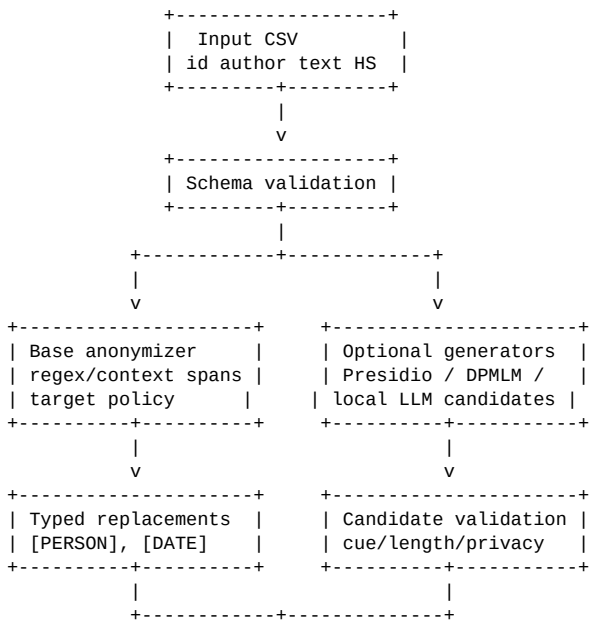
Default development output:

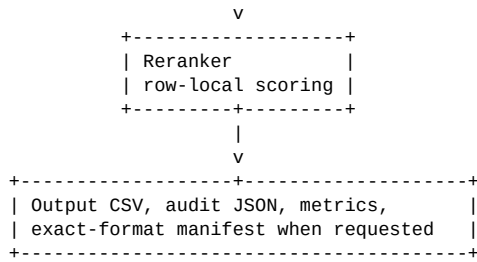
Output	Description
CSV with `privatized_text`	Original columns plus a new privatized text column.
Audit JSON	Row IDs, transformation metadata, aggregate metrics; no raw text needed for standard reports.
Metrics JSON	Local privacy/utility proxy metrics.

Official submission output:

Output	Description
Exact-format CSV	Same columns and order as input; selected text columns replaced in place.
Manifest JSON	Command, git commit, input/output hashes, row count, columns, mode, metrics, validation report.

Architecture





Core Modes

Mode	Behavior	Intended use
`utility`	Conservative masking.	Diagnostic comparison.
`balanced`	Default direct/quasi identifier masking while preserving target groups.	First official submission candidate.
`privacy`	More aggressive; target groups may be generalized.	Ablation or privacy-heavy alternate.
`balanced --style-scrub`	Balanced masking plus deterministic style normalization.	Authorship-risk pressure when utility remains stable.
`rerank-candidates`	Generates several candidates per row and chooses the best tradeoff.	Strongest alternate after first official score.
`rerank-candidates --presidio-augment`	Adds filtered Presidio entity spans as a candidate source.	Strongest current local experimental alternate.

What Gets Masked

Typed placeholders are used so downstream readers can understand what type of information was removed without seeing the raw value.

Entity type	Sources	Replacement	Example input	Example output
Person name	Context patterns; filtered Presidio optionally	`[PERSON]`	`My name is Amy Smith.`	`My name is [PERSON].`
Alias	`alias`, `aka`, `known as`, `goes by` context	`[ALIAS]`	`aka bluefox99`	`aka [ALIAS]`
User handle	Regex	`[USER]`	`ask @bluefox99`	`ask [USER]`
Email	Regex	`[EMAIL]`	`a.person@example.com`	`[EMAIL]`
Phone	Regex	`[PHONE]`	`+1 202 555 0199`	`[PHONE]`
URL	Regex	`[URL]`	`https://example.org/u/7`	`[URL]`
IP / row-like ID	Regex	`[ID]`	`ticket-AB12`	`[ID]`
Date	Regex; filtered durable Presidio dates optionally	`[DATE]`	`12/04/2024`	`[DATE]`
Location	Context patterns; filtered Presidio optionally	`[LOCATION]`	`near London`	`near [LOCATION]`
Organization/school	Regex	`[ORG]`	`Green Hill University`	`[ORG]`
Age	Regex	`[AGE]`	`I am 17 years old`	`[AGE]`
Target group	Target lexicon only when generalization is enabled	`[TARGET_GROUP:category]`	`immigrants should leave`	`[TARGET_GROUP:nationality_or_origin] should leave`

Default `balanced` mode preserves target-group words. This is intentional: hate-speech detection often needs to know who is targeted.

What Gets Preserved

The system preserves these by default:

- hate-speech target group wording;
- hostile action terms such as `ban`, `deport`, `attack`, `exclude`, `kill`;
- dehumanizing or hateful descriptors tracked as utility cues;
- negation and modality such as `not`, `never`, `should`, `must`;
- row IDs, labels, authors, and metadata columns;
- CSV row order and row count.

Preservation is a safety rule, not a claim that the lexicons fully define hate speech. The lexicons prevent privacy mechanisms from deleting obvious utility signals.

Dictionaries And Rule Sources

Resource	File	Used by	Purpose
`TARGET_GROUP_TERMS`	`privhsd/detectors.py`	target detection, target generalization, style protection, cue checks, DPMLM protection, weak token labels	Preserve or explicitly generalize identity target cues.
`UTILITY_CUES`	`privhsd/metrics.py`	metrics, reranking, Presidio filtering, LLM prompts, style protection, cue checks, DPMLM protection	Preserve action/dehumanization/threat context needed for HSD utility.
`ACTION_TERMS`	`privhsd/style.py`	style protection, cue checks, weak token labels, DPMLM protection	Preserve hostile action meaning.
`NEGATION_MODALITY_TERMS`	`privhsd/style.py`	cue checks, weak token labels, DPMLM protection	Preserve polarity and obligation meaning.
Regex patterns	`privhsd/detectors.py`	base anonymizer, metrics	Detect emails, URLs, handles, phones, dates, IDs, organizations, ages.
Context patterns	`privhsd/detectors.py`	base anonymizer	Detect names, aliases, and locations in simple local context.
Presidio filter lists	`privhsd/presidio_augment.py`	optional Presidio candidate	Reject false person terms, transient dates, `NRP`, and protected cue overlaps.

There is no broad hand-written misspelling dictionary. The current approach is narrower:

- repeated letters are normalized by the style scrubber when safe;
- repeated-letter tokens are frozen for DPMLM because they may be abuse or target cues;
- cue checks operate on phrase/term retention rather than spelling correction;
- weak token labels can mark style-like tokens for future reranking features.

Candidate Generation

Each row can have multiple candidate outputs. The reranker scores candidates and selects one row-local winner.

Candidate	Generated how	Main benefit	Main risk	Current status
`balanced`	Deterministic anonymizer in balanced mode.	Stable, auditable, low dependency.	May miss names/locations without context.	First official candidate.
`style_scrubbed`	`balanced` plus deterministic style scrub.	Reduces author style cues.	Can lower readability or character similarity.	Useful alternate.

Candidate	Generated how	Main benefit	Main risk	Current status
`privacy`	Deterministic anonymizer in privacy mode.	More privacy pressure.	Can overgeneralize target terms.	Ablation/alternate only.
`target_generalized`	Balanced mode with target generalization enabled.	Tests privacy-heavy target handling.	Often hurts HSD interpretability.	Candidate only.
`presidio_augmented`	Filtered Presidio spans added to balanced masking.	Catches extra names, locations, durable dates.	Presidio raw output has high false-positive risk.	Strongest local alternate after filtering.
`rewrite:<candidate_col>`	Precomputed external candidate column, e.g. DPMLM or LLM.	Allows model-backed rewrites.	Semantic drift, cue loss, new identifiers.	Must pass shared validation and reranking.

Reranking score rewards target cue retention, utility cue retention, character retention, and accepted Presidio spans. It penalizes residual identifiers, style risk, target/cue loss, length drift, semantic drift, and optional author-classifier confidence when an author column is usable.

External rewrite candidates are filtered before scoring. The shared validation rejects candidates that drop target/action cues, leave residual direct or quasi-identifiers, introduce new identifier signals, increase style risk, or drift too far in length/character similarity. Rejected candidates are recorded in the audit JSON by column and reason without storing raw text.

Optional Model Paths

Presidio

Presidio is not used raw. The filter accepts only mapped `PERSON`, `LOCATION`, and durable `DATE\_TIME` spans. It rejects:

- `NRP` spans because these often overlap with nationality/religion/political target cues;
- spans overlapping target or utility cues;
- person spans with risky shape;
- locations with risky shape;
- transient dates such as `today`, `tomorrow`, `yesterday`, and `christmas`;
- unsupported entity types.

Presidio is useful because the local deterministic detector can miss names or places. It is risky because hate-speech target terms often look like national, religious, or political entities. Therefore Presidio is a candidate source, not an unconditional replacement. For official alternates, route it through `rerank-candidates --replace-text --presidio-augment`, review the audit, and then run exact-format submission validation.

DPMLM

DPMLM is implemented as `generate-dpmlm-candidates`, not as direct submission output. It:

- uses `FacebookAI/roberta-base` by default;
- tokenizes text and computes rewrite-eligible tokens;
- freezes target terms, utility cues, action terms, negation/modality terms, stopwords, capitalized tokens, repeated-letter tokens, placeholders, and punctuation;

- rewrites only a small number of high-risk non-protected tokens;
- validates target retention, utility retention, HSD cue retention, character retention, length drift, new identifier signal, and style-risk increase;

- writes accepted candidates to a helper column for reranking.

Current local evidence does not justify DPMLM as a submission path: bounded reranking selected zero DPMLM candidates over deterministic alternatives.

Local LLM

The local LLM path talks to an OpenAI-compatible server such as LM Studio. It:

- sends a JSON-schema constrained prompt;
- asks for one replacement text only;
- tells the model to preserve target groups, hate/action cues, negation, threats, modality, and core meaning;
- parses JSON robustly;
- rejects candidates that lose target/utility cues or exceed length drift;
- writes accepted candidates to a helper column for reranking.

Current local LLM candidates are low-yield and should not be scaled without a better accepted-and-selected rate.

Evaluation And Audit

Check	Command	What it answers
Local proxy metrics	<code>`evaluate`</code>	Did identifier counts drop while cue/character retention stayed high?
Utility benchmark	<code>`benchmark-utility`</code>	Does a simple HSD classifier change behavior on privatized text?
HSD cue checks	<code>`check-hsd-cues`</code>	Were target/action/negation/modality terms preserved?
Metadata leakage	<code>`check-metadata-leakage`</code>	Do <code>`id`</code> or <code>`author`</code> values appear in original or privatized text?
Author risk	<code>`evaluate-author-risk`</code>	Can a local author classifier still identify authors after privatization?
HF utility probes	<code>`evaluate-hf-utility`</code>	Do optional pretrained HSD/toxicity probes drift?
Submission validation	<code>`validate-submission`</code>	Are row count, row order, columns, IDs, and metadata preserved?

Local metrics are not the official formula. They are development checks. The official target should be interpreted as:

```
high Utility_protected / Utility_original
low Privacy_protected / Privacy_original
```

Until official files and the official evaluator are available, further local optimization has limited value.

Synthetic Examples

Example 1: Direct Identifiers With HSD Cue Preservation

Input:

My name is Amy Smith and I live near London. Muslims should leave.

Balanced output:

My name is [PERSON] and I live near [LOCATION]. Muslims should leave.

Reason: `Amy Smith` and `London` are privacy-bearing. `Muslims should leave` is preserved because it contains target and action cues needed for HSD.

Example 2: Style Scrubbing

Input:

ask @bluefox99 on 12/04/2024!!! lol foreigners should leave %%%

Balanced output:

ask [USER] on [DATE]!!! lol foreigners should leave %%%

Balanced plus style scrub:

ask [USER] on [DATE]! [STYLE] foreigners should leave [SYMBOLS]

Reason: the handle and date are masked. Repeated punctuation, idiolect markers, and symbol bursts are style cues and can be normalized without removing the HSD cue.

Example 3: Explicit Privacy Mode

Input:

immigrants should leave this town.

Privacy or target-generalized output:

[TARGET_GROUP:nationality_or_origin] should leave this town.

Reason: target generalization is available for privacy-heavy experiments, but it is not the default because target identity may be necessary for HSD utility.

Current Verdict

Question	Answer
Are we done locally?	Mostly yes until official data arrives.
First official run?	`create-submission --replace-text --mode balanced`.
Strongest alternate?	`rerank-candidates --replace-text --presidio-augment`, then exact-format validation.
Should we do more DPMLM now?	No, unless official scores show a reason.
Should we train a new model now?	No, not without official data and a clear metric gain target.
What should a tutor review?	Reranking objective, privacy/utility metrics, author-risk evaluation, and whether the protected cue policy is defensible.

Next Steps When Real Data Arrives

1. Inspect schema and identify text, ID, author, and label columns.
2. Run metadata leakage checks on original text for `id` and `author`.
3. Create a first exact-format `balanced` submission.
4. Validate row count, row order, columns, IDs, labels, authors, and metadata.
5. Run official evaluator and record Utility and Privacy ratios.
6. If an author column has repeated authors, run `evaluate-author-risk`.
7. Run `rerank-candidates --replace-text --presidio-augment` as the strongest alternate.
8. Submit the alternate only if exact validation passes and official scoring is better.
9. Revisit DPMLM/LLM only if official privacy remains weak and utility headroom exists.

Codebase Audit Prompt

Use this prompt for another model, teammate, or PhD tutor:

You are auditing the PrivHSD codebase at `/home/bati/projects/PrivHSD-Challenge`.

Goal:

Assess whether the pipeline design is technically sound for a privacy-preserving hate-speech-detection challenge where the output must reduce author-identifying signals while preserving HSD utility.

Start by reading:

1. docs/privhsd_system_design_paper.md
2. docs/methodology_justification.md
3. docs/dp_text_privacy_literature_notes.md
4. docs/experiment_verdict.md
5. docs/pipeline_design.md
6. agents/task_board.md

Then inspect these implementation modules:

1. privhsd/pipeline.py
2. privhsd/detectors.py
3. privhsd/style.py
4. privhsd/metrics.py
5. privhsd/rerank.py
6. privhsd/presidio_augment.py
7. privhsd/dpmlm_candidates.py
8. privhsd/local_llm.py
9. privhsd/submission.py
10. privhsd/author_risk.py
11. privhsd/metadata_leakage.py

Audit questions:

1. Does the document accurately describe the code behavior?
2. Are target/action/negation cues protected enough for HSD utility?
3. Are any dictionaries unjustified, too broad, or likely to overfit Dynahate?
4. Does reranking optimize a defensible privacy/HSD tradeoff?
5. Are Presidio, DPMLM, and LLM candidates sufficiently constrained?
6. What privacy risks remain if direct identifiers are removed?
7. What should wait until official `id,author,text,HS` data is available?
8. What is the smallest next implementation step with likely official-metric value?

Expected output:

- A short verdict table.
- Top 5 technical risks.
- Top 5 recommended changes, separated into "do now" and "wait for official data".
- Any mismatch between docs and code, with file/line references.

Appendix: Main Commands

First exact-format candidate:

```
python -m privhsd.cli create-submission \
```



```
--input INPUT.csv \  
--output data/outputs/SUBMISSION.balanced.csv \  
--text-col text \  
--id-col id \  
--replace-text \  
--mode balanced \  
--manifest data/outputs/SUBMISSION.balanced.manifest.json
```

Strongest local alternate:

```
python -m privhsd.cli rerank-candidates \  
--input INPUT.csv \  
--output data/outputs/SUBMISSION.reranked_presidio.csv \  
--text-col text \  
--id-col id \  
--replace-text \  
--presidio-augment \  
--audit data/outputs/SUBMISSION.reranked_presidio.audit.json
```

Metadata leakage check:

```
python -m privhsd.cli check-metadata-leakage \  
--input data/outputs/SUBMISSION.balanced.csv \  
--text-col text \  
--metadata-col id \  
--metadata-col author \  
--id-col id \  
--output data/outputs/SUBMISSION.metadata_leakage.json
```

Author-risk check when `author` has repeated rows:

```
python -m privhsd.cli evaluate-author-risk \  
--input data/outputs/SUBMISSION.with_original_and_private.csv \  
--text-col text \  
--privatized-col privatized_text \  
--author-col author \  
--id-col id \  
--label-col HS \  
--output data/outputs/SUBMISSION.author_risk.json
```